

3rd Day Internship (9-June) Internship CoE – MathWorks DeepTech Labs

https://in.mathworks.com/help/matlab/language-fundamentals.html?s_tid=CRUX_lftnav

Name: HUJEFA

Reg.no: 2023BECE07AED033

Semester & Branch: VII Sem ECE 2023-27

Institution: Alliance University, Bengaluru

Operators and Elementary Operations

```
%% Array vs Matrix Operations
```

```
% Matrix operations follow linear algebra rules.  
% Array operations perform element-wise calculations.
```

```
A = [1 2; 3 4]; B = [5 6; 7 8];
```

```
C = A * B      % Matrix multiplication
```

```
C = 2x2  
    19    22  
    43    50
```

```
D = A .* B      % Element-wise multiplication
```

```
D = 2x2  
     5    12  
    21    32
```

```
E = A .^ 2      % Element-wise power
```

```
E = 2x2  
     1     4  
     9    16
```

```
%% Compatible Array Sizes % Arrays can be combined if their sizes  
are compatible.
```

```
A = [1 2 3];
```

```
B = 10;
```

```
C = A + B      % Scalar expansion
```

```
C = 1x3
```

11 12 13

```
D = [4 5 6]; E = A + D      % Arrays  
with same size
```

```
E = 1×3  
     5     7     9
```

```
%% Operator Precedence % MATLAB follows a specific order while  
evaluating expressions.
```

```
result1 = 2 + 3 * 4      % Multiplication happens first
```

```
result1 =  
14
```

```
result2 = (2 + 3) * 4      % Parentheses have higher priority
```

```
result2 =  
20
```

```
%% Floating-Point Numbers % MATLAB stores numeric values as  
double precision by default. A = 3.14159; class(A)      %  
Check data type
```

```
ans =  
'double'
```

```
B = single(3.14159); class(B)      %  
Single precision data type
```

```
ans =  
'single'
```

```
%% Integer Data Types % Integer types store whole numbers  
efficiently.
```

```
A = int8(100);  
B = uint16(500); class(A)
```

```
ans =  
'int8'
```

```
class(B)
```

```
ans =  
'uint16'
```

Relational Operators

```
A = 5; B = 10;
```

```
A == B      % Check equality
```

```
ans =  
logical  
0
```

```
A ~= B      % Check inequality
```

```
ans =  
logical  
1
```

```
A > B      % Greater than
```

```
ans =  
logical  
0
```

```
A < B      % Less than
```

```
ans =  
logical  
1
```

```
A >= 5     % Greater than or equal to
```

```
ans =  
logical  
1
```

```
B <= 10     % Less than or equal to
```

```
ans =  
logical  
1
```

```
% == -> Equal to
% ~= -> Not equal to
% > -> Greater than
% < -> Less than
% >= -> Greater than or equal to
% <= -> Less than or equal to
```

```
%% Compare Arrays
```

```
A = [1 2 3]; B = [1 0 3];
```

```
A == B      % Compare elements individually
```

```
ans = 1x3 logical array
1    0    1
```

```
% Relational operators return logical values (1 = true, 0 = false).
```

```
%% isequal Function
```

```
A = [1 2 3]; B = [1 2 3]; isequal(A,B)      % Check if two arrays
are completely equal
```

```
ans =
logical
1
```

```
% isequal() returns true only if size and elements are identical.
```

```
%% Array Comparison marks = [85 42 67 91
38]; result = marks >= 50      % Find passing
marks
```

```
result = 1x5 logical array
1    0    1    1    0
```

```
%% Operator Precedence result = 5 + 2 * 3      %
```

```
Multiplication happens first
```

```
result =
11
```

```
result2 = (5 + 2) * 3    % Parentheses have higher priority
```

```
result2 =  
21
```

Logical (Boolean) Operations

```
%% Logical AND (&)
```

```
A = [1 2 3 4 5];
```

```
A > 2 & A < 5    % Elements greater than 2 AND less than 5
```

```
ans = 1x5 logical array  
0    0    1    1    0
```

```
%% Logical OR (|)
```

```
A = [1 2 3 4 5];
```

```
A < 2 | A > 4    % Elements less than 2 OR greater than 4
```

```
ans = 1x5 logical array  
1    0    0    0    1
```

```
%% Logical NOT (~)
```

```
A = [1 0 1 0];
```

```
~A
```

```
ans = 1x4 logical array  
0    1    0    1
```

```
% ~ reverses logical values (1 becomes 0 and vice versa).
```

```
%% Exclusive OR (xor)
```

```
xor(true,false)
```

```
ans =  
logical  
1
```

```
xor(true,true)% xor returns true only if exactly one input is true.
```

```
ans =  
logical  
0
```

```
%% all Function
```

```
A = [1 1 1];
```

```
all(A)
```

```
ans =  
logical  
1
```

```
% all returns true if all elements are nonzero or true.
```

```
%% any Function
```

```
A = [0 0 1 0];
```

```
any(A)
```

```
ans =  
logical  
1
```

```
%% find Function A
```

```
= [10 25 40 15];
```

```
find(A > 20)
```

```
ans = 1×2  
      2      3
```

```
% find returns indices of elements satisfying a condition.  
% true represents logical 1.  
% false represents logical 0.
```

```
%% logical and islogical
```

```
A = logical([1 0 2])
```

```
A = 1×3 logical array  
      1      0      1
```

```
islogical(A)
```

```
ans =  
logical  
1
```

```
% logical converts numeric values to logical values.  
% islogical checks whether input is logical.
```

Set Operations

```
%% union Function
```

```
A = [1 2 3];  
B = [3 4 5]; C = union(A,B)      % Combine sets and  
  
remove duplicates
```

```
C = 1×5  
    1     2     3     4     5
```

```
% union() returns all unique elements from both sets.
```

```
%% intersect Function
```

```
A = [1 2 3 4];  
B = [3 4 5 6]; C = intersect(A,B) % Find  
  
common elements
```

```
C = 1×2  
    3     4
```

```
% intersect() returns elements present in both sets.
```

```
%% setdiff Function
```

```
A = [1 2 3 4]; B = [3 4 5];  
  
C = setdiff(A,B)      % Elements in A but not in B
```

```
C = 1×2  
    1     2
```

```
% setdiff() returns elements unique to the first set.
```

```
%% setxor Function
```

```
A = [1 2 3 4];  
B = [3 4 5 6];  
C = setxor(A,B)      % Elements present in only one set
```

```
C = 1×4  
    1     2     5     6
```

```
% setxor() returns elements that are not common to both sets.
```

```
%% ismember Function
```

```
A = [2 5 7]; B = [1 2 3 4 5];
```

```
TF = ismember(A,B)      % Check membership
```

```
TF = 1x3 logical array  
    1    1    0
```

```
% ismember() returns logical values indicating membership.
```

```
%% unique Function
```

```
A = [1 2 2 3 4 4 5]; B = unique(A)      % Remove  
duplicate values
```

```
B = 1x5  
    1    2    3    4    5
```

```
% unique() returns distinct values from an array.
```

Bit Wise Operations.

```
%% bitand Function
```

```
A = 12;      % Binary: 1100
```

```
B = 10;      % Binary: 1010
```

```
C = bitand(A,B)      % Bit-wise AND
```

```
C =  
8
```

```
% bitand() returns 1 only if both bits are 1.
```

```
%% bitor Function
```

```
A = 12;
```

```
B = 10;
```

```
C = bitor(A,B)      % Bit-wise OR
```

```
C =  
14
```



```
% bitor() returns 1 if at least one bit is 1.
```

```
%% bitxor Function
```

```
A = 12;  
B = 10; C = bitxor(A,B)      % Bit-wise XOR
```

```
C =  
6
```

```
% bitxor() returns 1 only when bits are different.
```

```
%% bitcmp Function
```

```
A = uint8(10);  
  
B = bitcmp(A)      % Bit-wise complement
```

```
B =  
uint8  
245
```

```
% bitcmp() flips all bits (0 becomes 1 and 1 becomes 0).
```

```
%% bitget Function A = 13;          %  
  
Binary: 1101 bit = bitget(A,1)    % Get least  
significant bit
```

```
bit = 1
```

```
% bitget() extracts the bit at the specified position.
```

```
%% bitset Function
```

```
A = 8;          % Binary: 1000  
B = bitset(A,1) % Set first bit to 1
```

```
B =  
9
```

```
% bitset() changes the bit at a specified position to 1.
```

Special Characters

```
%%Equal Sign (=)

A = 10;      % Assign value to a variable %

= is used to assign values to variables.

%% Semicolon (;)

A = 5;

% Suppress output in Command Window % ;

prevents MATLAB from displaying output.

%% Colon Operator (:) A = 1:5      % Create
a vector from 1 to 5
```

```
A = 1x5
    1     2     3     4     5
```

```
% : is used for vector creation and indexing.
```

```
%% Square Brackets [ ]
```

```
A = [1 2 3;
     4 5 6]      % Create a matrix
```

```
A = 2x3
    1     2     3
    4     5     6
```

```
% [ ] are used to create and concatenate arrays.
```

```
%% Parentheses ( ) A = [10 20 30]; value
= A(2)      % Access second element
```

```
value = 20
```

```
% ( ) are used for indexing and function inputs.
```

```
%% Percent Symbol (%) % This is
```

```
a single-line comment.
```

```
A = 10; % % is used to write comments
```

```
in MATLAB.
```

```
%% Block Comments
```

```
%{ This is a block  
comment.
```

```
It spans multiple lines.
```

```
%} % %{ %} are used for multi-line
```

```
comments. %% Element-wise Operations (.)
```

```
A = [1 2 3];
```

```
B = A .* A
```

```
B = 1x3
```

```
1      4      9
```

```
%% Line Continuation (...)
```

```
A = 1 + 2 + 3 + ...  
    4 + 5
```

```
A =
```

```
15
```

```
% ... continues a statement on the next line.
```

```
%% Cell Arrays { }
```

```
C = {10, 'MATLAB', true}
```

```
C = 1x3 cell
```

	1	2	3
1	10	'MATLAB'	1

```
% { } are used to create cell arrays.
```

```
%% if-elseif-else Statement
```

```
marks = 75;

if marks >= 90
    grade = 'A';
elseif marks >= 60
    grade = 'B';
else
    grade = 'C'; end disp(grade) % if-elseif-else
executes code based on conditions.
```

```
%% switch-case Statement
```

```
day = 2;

switch day
case 1
    disp("Monday")
case 2
    disp("Tuesday")
otherwise
    disp("Invalid Day")
end % switch selects one block of code from multiple
options.
```

```
%% for Loop
```

```
for i = 1:5     disp(i) end % for loop repeats code a
fixed number of times.
```

```
%% while Loop
```

```
count = 1;

while count <= 5     disp(count)     count = count + 1; end %
while loop executes as long as the condition is true.
```

```
%% break Statement
```

```
for i = 1:10      if i == 5      break      end
disp(i) end % break immediately exits the loop.
```

```
%% continue Statement
```

```
for i = 1:5
if i == 3
continue
    end    disp(i) end % continue
skips the current iteration.
```

```
%% pause Statement disp("Start") pause(2)

% Pause for 2 seconds disp("End") % pause
temporarily stops MATLAB execution.
```

```
%% return Statement
```

```
disp("Before Return")

return
```

```
disp("This line will not execute") % return
exits the current script or function.
```